

Übungsblatt 7: Moderne CPU-Architekturen

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen Vorlesung 7: Datenabhängigkeiten und Superskalarität, Register Renaming / ROB, Branch Prediction und Spekulation (inkl. Spectre) sowie Compiler-Optimierungen in Godbolt (-01 vs. -03).

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 8 P, Aufgabe 2: 12 P, Aufgabe 3: 4 P (Gesamt: 24 P)

Teil 1: Theorie — Von In-Order zu Out-of-Order

Aufgabe 1.1: Datenabhängigkeiten und Superskalarität

Superskalare Prozessoren nutzen Instruction-Level Parallelism (ILP) über mehrere Execution Units. Das größte Hindernis sind Datenabhängigkeiten.

Gegeben:

```
1: lw  t0, 0(s1)
2: add t1, t0, t2
3: sub t0, t4, t5
4: and t6, t0, t3
```

1. Identifizieren Sie eine *True Dependency* (Read-After-Write / RAW). Zwischen welchen Befehlen tritt sie auf, und warum kann die Hardware sie nicht parallelisieren?
2. Zwischen den Befehlen 2 und 3 liegt eine „falsche“ Abhängigkeit (Write-After-Read / WAR). Warum? Welches fatale Problem entstünde, wenn eine einfache CPU Befehl 3 vor Befehl 2 ausführen würde?

Musterlösung Aufgabe 1.1

1. **RAW** zwischen Befehl 1 (`lw t0, ...`) und Befehl 2 (`add t1, t0, ...`): Befehl 2 braucht den Wert in `t0`. Die Abhängigkeit ist real; ohne Ergebnis von 1 ist 2 nicht korrekt ausführbar.
2. **WAR** zwischen 2 und 3: Befehl 3 ist mathematisch unabhängig von 2, überschreibt aber dasselbe Architektur-Register `t0`. Würde Befehl 3 *schreiben*, bevor Befehl 2 den alten Wert aus `t0` *gelesen* hat, läse 2 den *neuen* Wert von `t0` statt den von `lw` — falsches Additions-Ergebnis. Die Schreib-/Lese-Reihenfolge auf dem Architektur-Register muss also gewahrt bleiben (oder per Renaming entkoppelt werden).

Aufgabe 1.2: Register Renaming und der ROB

Out-of-Order-(OoO-)CPUs lösen falsche Abhängigkeiten und führen Befehle asynchron aus.

1. Erklären Sie *Register Renaming*. Wie zerstört die Technik die WAR-Abhängigkeit zwischen Befehl 2 und 3 (Architektur- vs. physische Register)?
2. Warum sind Reorder Buffer (ROB) und *In-Order Commit* zwingend, um das System bei Interrupts vor inkonsistenten Zuständen zu schützen?

Musterlösung Aufgabe 1.2

1. Intern gibt es einen großen Pool physischer Register; die 32 RISC-V-Architektur-Register werden dynamisch darauf abgebildet. Für den Schreibvorgang von Befehl 3 erhält `t0` ein *neues* physisches Register (z. B. `t0` $\rightarrow$ `P11`), während Befehl 2 weiter aus dem alten Mapping (`P10`) liest. Schreibziele sind physisch getrennt $\rightarrow$ WAR verschwindet; beide können im selben Issue-Takt auf zwei ALUs laufen.
2. OoO führt aus, sobald Operanden bereit sind — spätere Befehle können fertig sein, während frühere noch auf RAM warten. Ein Interrupt mittendrin trafe einen inkonsistenten Zustand. Der ROB hält Ergebnisse spekulativ und *committet* streng in Programmreihenfolge in Architektur-Register/RAM — nach außen erscheint eine sequentielle CPU.

Teil 2: Theorie — Branch Prediction und Spekulation

Aufgabe 2.1: Das Problem tiefer Pipelines

Moderne Prozessoren haben oft Pipelines mit weit über 15 Stufen.

1. Erklären Sie das *Control-Hazard*-Problem bei Sprüngen (`beq`). Warum führen Fehlprognosen bei tiefen Pipelines zu massiven Leistungseinbrüchen (*Misprediction Penalty*)?

Musterlösung Aufgabe 2.1

Bei `beq` weiß die Fetch-Stufe noch nicht, ob der Sprung genommen wird — die ALU entscheidet erst später. Die Pipeline muss aber jeden Takt füttern und spekuliert. Bei z. B. 15 Stufen können viele spekulative Befehle im Fließband liegen. Bei Fehlprognose: Flush → *Misprediction Penalty* (viele verlorene Takte, IPC-Einbruch).

Aufgabe 2.2: Branch History und Spekulation

1. Skizzieren Sie einen *2-Bit saturierenden Zähler*. Welchen Vorteil hat er gegenüber einem 1-Bit-Prädiktor bei verschachtelten Schleifen (Hysterese)?
2. Erklären Sie *spekulative Ausführung*. Was tut die CPU, während sie auf die ALU-Bestätigung des Sprungs wartet?

Musterlösung Aufgabe 2.2

1. Vier Zustände: 00 Strongly NT, 01 Weakly NT, 10 Weakly T, 11 Strongly T. Taken → +1 (cap 11), Not Taken → -1 (floor 00). **Hysterese:** Nach langer Taken-Schleife (11) und einem Abbruch fällt der Zähler nur auf 10 — beim nächsten Schleifenstart sofort wieder korrekt „Taken“. Ein 1-Bit-Prädiktor hätte nach dem Abbruch komplett umgeschaltet und falsch geraten.
2. Der Predictor wählt einen Pfad; die CPU wartet nicht passiv, sondern schiebt die vorhergesagten Instruktionen durch Renaming in die Execution Units und rechnet/lädt „auf Verdacht“. Bestätigt sich die Vorhersage, ist die Arbeit schon getan (höherer IPC).

Aufgabe 2.2b: 2-Bit-Prädiktor — Zustandstabelle

Ein 2-Bit saturierender Zähler startet im Zustand 11 (*Strongly Taken*). Kodierung: 00 Strongly NT, 01 Weakly NT, 10 Weakly T, 11 Strongly T. Vorhersage: Zustände 1x → Taken, 0x → Not Taken.

Gegeben die tatsächliche Sprungsequenz: **T, T, NT, NT, T**.

1. Füllen Sie eine Zustandstabelle: für jeden Schritt Startzustand, Vorhersage, tatsächliches Ergebnis, Treffer/Miss, Folgezustand.
2. Wie viele Treffer (korrekte Vorhersagen) ergeben sich insgesamt?

Musterlösung Aufgabe 2.2b

Schritt	Start	Vorhersage	Ist	Treffer?	Folge
1	11	T	T	ja	11
2	11	T	T	ja	11
3	11	T	NT	nein	10

Schritt	Start	Vorhersage	Ist	Treffer?	Folge
4	10	T	NT	nein	01
5	01	NT	T	nein	10

Treffer: 2 von 5 (Schritte 1 und 2).

Aufgabe 2.3: Seitenkanalangriffe (Spectre)

1. Bei Misprediction macht der ROB einen *Rollback* (Squash). Welcher Zustand wird dabei typisch *nicht* gelöscht und öffnet die Tür für Spectre?
2. Wie nutzt ein Angreifer Timing als Orakel, um eigentlich unerreichbare Daten zu rekonstruieren?

Musterlösung Aufgabe 2.3

1. Rollback verwirft spekulative Register-Updates und unterbindet spekulative Stores in den RAM. Spekulativ geladene Daten hinterlassen aber Spuren in den **Cache-Lines** (z. B. L1) — die bleiben.
2. Der Angreifer zwingt die CPU spekulativ zu einem Array-Zugriff, dessen Index von einem „Geheimnis“ abhängt. Nach dem Rollback misst er die Zugriffszeit auf seine Array-Elemente: der schnelle Hit verrät die Position und damit das Geheimnis.

Teil 3: Praxis — Performance-Analyse in Godbolt

Szenario: Der Compiler kann Code für die Pipeline vorbereiten. Nutzen Sie [Compiler Explorer \(godbolt.org\)](https://godbolt.org).

Gegeben:

```
void vector_add(int *a, int *b, int *c, int len) {  
    for (int i = 0; i < len; i++) {  
        c[i] = a[i] + b[i];  
    }  
}
```

Aufgabe 3.1: Instruction Scheduling und Loop Unrolling

Öffnen Sie Godbolt, Compiler **RISC-V rv32gc gcc**, Code einfügen.

1. Kompilieren Sie mit `-O1`. Wo liegen typisch die `lw`- und `add`-Instruktionen in der Schleife?
2. Wechseln Sie auf `-O3`. Beobachten Sie:
 - *Instruction Scheduling*: Werden `lw/sw` so umsortiert, dass Load-Use-Stalls gemildert werden?
 - *Loop Unrolling*: Warum wird die Assembler-Ausgabe länger — und warum hilft das der Branch Prediction?

Dokumentieren Sie Ihre Beobachtungen in wenigen Sätzen.

Musterlösung Aufgabe 3.1 (exemplarisch; Details je nach Godbolt-/GCC-Version)

-O1: Die Schleife bleibt nah am C-Original: typisch `lw` für `a[i]/b[i]`, dann zeitnah `add`, dann `sw`; viele Branches, da jedes Element eine Iteration ist.

-O3:

- *Scheduling*: Oft mehrere `lw` für unterschiedliche Indizes vor dem ersten `add` — Wartezeit auf den ersten Load wird mit dem nächsten Load gefüllt (weniger Load-Use-Stalls).
- *Unrolling*: Mehrere Index-Schritte (z. B. 4 oder 8) pro Schleifenkörper → weniger Kontroll-Overhead (`addi/bne`). Weniger Branches → selteneres Flush-Risiko für die Prediction.